

Lab Sheet 8: Software Testing & CI/CD

Objectives

By completing today's workshop, you will:

- Write functional **Unit Tests** for your backend application logic.
 - Implement **Integration Tests** to verify your API routes and Dockerized service communication.
 - Automate your testing strategy by building a **Continuous Integration (CI)** pipeline using GitHub Actions.
 - Architect an isolated **Test Container** to run tests without altering development data.
 - Integrate your test container into the template repository's existing CI workflow and Docker Compose setup.
-

Introduction: Securing your Codebase

In Assessment 1, you are building a critical system (a Robot Ground Control Station). You must ensure that users cannot bypass Role-Based Access Control (RBAC) and that invalid coordinates are not sent to the physical (or virtual) robot. Manual testing is too slow. Today, you will establish an automated testing safety net.

*Note: The examples below use Python and **pytest**. If you are building a Node.js, a C#, or any other language backend, you should adapt these concepts using **Jest/Mocha, MSTest**, etc.*

Task 1: Writing Unit Tests

Goal: Test small, isolated pieces of your application logic.

The Concept: Unit tests do not need a database or network connection. They test raw logic. Let's test a hypothetical coordinate validator.

Action:

1. In your backend codebase, create a new folder called `tests/`.
2. Create a file called `test_validation.py`.
3. Write a simple function and an accompanying test. Notice the "Arrange, Act, Assert" pattern:

```
# The logic (usually imported from your main code)
def is_valid_target(x, y):
    if x < 0 or x > 100 or y < 0 or y > 100:
        return False
    return True

# The test
def test_valid_coordinates():
    # Arrange (Setup inputs)
    good_x, good_y = 50, 50
    bad_x, bad_y = 150, -10

    # Act (Run the function)
    result_good = is_valid_target(good_x, good_y)
    result_bad = is_valid_target(bad_x, bad_y)

    # Assert (Check the outcomes)
    assert result_good == True
    assert result_bad == False
```

4. Run your test locally in the terminal (e.g., by typing `pytest`, `npm test`, or something else based on the language you are using). Ensure it passes.
5. Once this first simple example is working, write at least 6 more unit tests covering edge cases of some *actual functionalities* you have implemented in your backend code (e.g., coordinates at the boundaries, non-integer inputs, etc.).

Task 2: API Integration Testing

Goal: Test that your web server responds correctly to HTTP requests.

The Concept: Integration tests ensure that the routing and request handling work. For instance, if someone tries to hit your `/api/move` endpoint without an authentication token, it should reject them.

NOTE: the fastapi library is not installed on the lab PCs (and cannot be installed due to admin restrictions). However, this is installed inside the Docker container. You can write these tests in your local environment and they will run correctly inside the container during CI.

Action:

1. In your `tests/` folder, create `test_api.py`.
2. Write an integration test that hits your local application framework. (If using FastAPI/Flask in Python, you can use a `TestClient`. If using Express in Node, use `Supertest`).

```
from fastapi.testclient import TestClient
from main import app # Import your actual app

client = TestClient(app)
```

```
def test_unauthorized_move():  
    # Attempting to move without a token  
    response = client.post("/api/move", json={"x": 10, "y": 10})  
  
    # The API should block this and return a 401 Unauthorized status  
    assert response.status_code == 401
```

3. **Extend the Test:** Write a second test that passes a valid "Viewer" role token. Assert that the API returns a 403 Forbidden (because Viewers cannot move the robot).
4. Implement two additional tests for a feature of your choice (e.g., a successful move command with an "Operator" token, or a failed move due to invalid coordinates).

Task 3: Automating with GitHub Actions (CI/CD)

Goal: Ensure your tests run automatically every time you push code to GitHub.

The Concept: You have written tests, but humans forget to run them. Continuous Integration (CI) uses a server to spin up an environment, install your dependencies, and run the tests automatically on every commit.

Action:

1. In the root directory of your repository, create the following folder structure:
 .github/workflows/.
2. Inside that folder, create a file called ci-tests.yml.
3. Paste in the following YAML configuration. (Modify the setup steps if you are using Node.js instead of Python):

```
name: Automated Testing Pipeline  
  
# Trigger this workflow on every push to the main branch  
on:  
  push:  
    branches: [ "main" ]  
  pull_request:  
    branches: [ "main" ]  
  
jobs:  
  test:  
    runs-on: ubuntu-latest  
    steps:  
      - name: Check out repository code  
        uses: actions/checkout@v3  
  
      - name: Set up Python  
        uses: actions/setup-python@v4  
        with:  
          python-version: "3.10"  
  
      - name: Install dependencies  
        run: |  
          pip install pytest fastapi  
          # Add any other requirements.txt installations here  
  
      - name: Run test suite  
        run: |  
          pytest
```

4. Commit and push this file to GitHub.
5. Go to your GitHub repository in the browser, click the **Actions** tab, and watch your pipeline build and test your code automatically!

Task 4: Test Coverage Reporting

Goal: Generate a report detailing exactly what percentage of your codebase is covered by tests.

The Concept: Code coverage tools analyze your application while the tests run to see which lines of code were executed and which were ignored.

Action:

1. Install a coverage tool (e.g., `pytest-cov` for Python or `nyc` for Node.js).
2. Run your tests with the coverage flag: `pytest --cov=.`
3. Review the terminal output. It will tell you the percentage of code covered. Your goal as an engineer is generally to maintain $> 80\%$ coverage for critical systems.
4. *Bonus:* Add this coverage step to your GitHub Actions YAML file so the coverage report is generated automatically on the server!

Task 5: Architectural Upgrade - The Test Container

Goal: Create a dedicated Docker container specifically for executing your test suite safely.

The Concept: In Task 3, you ran tests directly on the GitHub server. However, last week you learned about Containerisation. To ensure your tests run in the exact same environment as production, we will create a Docker Compose override file that spins up a special `test-runner` container.

Action:

1. In the **root** of your repository (where your main `docker-compose.yml` lives), create a new file named `docker-compose.test.yml`.
2. Add the following configuration to define your Test Container:

```
version: '3.8'

services:
  test-runner:
    build: ./backend
    # Override the normal server command to run your test framework instead!
    command: pytest tests/ -v
    depends_on:
      - backend
```

3. **Test it locally:** In your terminal, run the following command to spin up your test container alongside your stack:

```
docker compose -f docker-compose.yml -f docker-compose.test.yml run --rm test-runner
```

4. You should see Docker build the container, execute your test files, print the results, and automatically destroy the container (`--rm`).

Task 6: Integrating into the Template's CI

Goal: Modify the `CMP9134_2526_template` repository's existing GitHub Actions pipeline to use your new Test Container.

The Concept: Your template repository from Week 7 likely already came with a workflow file. Let's upgrade it to use Docker Compose for testing instead of bare-metal Python.

Action:

1. In VS Code, navigate to the template's original CI YAML file (look inside `.github/workflows/`).
2. Scroll down to the `test` job where the containers are started.
3. Add a new step to run your test container using the Docker Compose command you tested in Task 5:

```
- name: Run Dockerized Test Suite
  run: |
    docker compose -f docker-compose.yml -f docker-compose.test.yml \
    run --rm test-runner
```

4. **Commit and Push:** Save your changes and push them to GitHub.
5. Go back to the **Actions** tab on GitHub. You should now see your tests running safely *inside* the containerized environment! If a test fails, the container will return an error code and properly block the CI pipeline.

Further Resources

- **PyTest Documentation:** docs.pytest.org - *The standard testing framework for Python.*
- **Jest Documentation:** jestjs.io - *A delightful JavaScript Testing Framework with a focus on simplicity.*
- **Docker Compose Overrides:** docs.docker.com/compose - *Learn how to merge multiple compose files for different environments.*